

Sistema de Alerta Temprana de Fatiga y Somnolencia Multimodal para Choferes

Tucno Ivan, Vizcardo Roberto, Ruiz Ricardo, Picon Yan

Facultad de Ingeniería Industrial y de Sistemas, Universidad Nacional Federico Villarreal Lima, Perú

I. Descripción del problema real

La incidencia de accidentes de tránsito causados por el cansancio físico y la somnolencia al volante representa uno de los problemas más graves en el transporte interprovincial y de carga pesada en el Perú. Debido a las rutas tan largas, la monotonía del viaje y los horarios nocturnos, los conductores sufren constantemente de un agotamiento extremo que disminuye sus reflejos. Esta condición del cuerpo da origen a los llamados microsueños, que son esos segundos en los que el chofer se queda completamente dormido frente al volante sin darse cuenta.

En la actualidad, los vehículos comerciales no cuentan con un sistema de monitoreo continuo dentro de la cabina que sea capaz de detectar esta falta de atención en tiempo real para avisarle al conductor de forma oportuna. La falta de un mecanismo de control automático hace que los accidentes en las carreteras sigan aumentando, lo que trae como consecuencia pérdidas de vidas humanas, daños en las vías y graves perjuicios económicos para las empresas de transporte.

II. Contexto del proyecto

Con el propósito de reducir este escenario de riesgo, este proyecto plantea el diseño de una solución tecnológica económica y de respuesta rápida que funcione dentro del vehículo sin depender de una conexión a internet. En nuestra carrera de ingeniería de sistemas, este enfoque se conoce como procesamiento local o en el borde.

Como la captura y el análisis continuo de video son tareas demasiado pesadas para que las haga un solo chip electrónico pequeño, se ha decidido utilizar una arquitectura de red local que divida el trabajo. Mediante este diseño, la tarjeta electrónica instalada en el tablero funciona como un servidor que transmite el video inalámbrico en tiempo real, mientras que una computadora portátil recibe y analiza las imágenes. Todo el programa se organiza usando

hilos de trabajo independientes, lo que garantiza que la alarma física suene en el momento exacto en que se detecta la somnolencia, sin retrasar ni congelar la transmisión de la cámara.

III. Beneficiarios del sistema

La implementación de esta tecnología ayuda de forma directa a los choferes de buses y camiones, quienes tendrán una herramienta de supervisión en su cabina que cuidará sus vidas ante el cansancio extremo. Al mismo tiempo, las empresas de transporte se ven favorecidas al disminuir los choques, las pérdidas de mercadería y los costos de reparaciones o seguros, mejorando el cuidado de sus trabajadores.

De forma indirecta, los pasajeros y todas las personas que viajan por las carreteras del país estarán mucho más seguros. Finalmente, a nosotros como estudiantes de ingeniería de sistemas, este proyecto nos permite ganar experiencia práctica muy valiosa. Nos da la oportunidad de aplicar temas del curso como la programación de servidores web en chips, la comunicación de red local, el trabajo con hilos en paralelo y la optimización de programas de visión artificial para resolver un problema de la sociedad.

IV. Objetivo general del proyecto

Diseñar e implementar un sistema distribuido de alerta temprana de fatiga para choferes que conecte una tarjeta electrónica y una computadora a través de una red Wi-Fi local, permitiendo analizar la apertura de los ojos en tiempo real y activar una alarma sonora física de forma inmediata al detectar un microsueño.

V. Objetivos específicos de adquisición y procesamiento

Con el fin de hacer posible el análisis de las imágenes de video, el primer objetivo específico

se enfoca en configurar la cámara de la tarjeta del tablero (ESP32-CAM) para capturar el rostro del conductor en una resolución ligera (QVGA) y transmitir las fotos de forma continua. Bajo esta perspectiva, se busca desarrollar un programa en la computadora usando el lenguaje Python para conectarse a esta transmisión de red, recibir las imágenes y aplicar un filtro que descarte una foto de cada dos, aliviando de manera drástica el esfuerzo del procesador de la computadora. Concluyendo esta etapa de procesamiento, se pretende aplicar la fórmula matemática de la Relación de Aspecto del Ojo (EAR) mediante la herramienta MediaPipe de Google para calcular la apertura promedio de los párpados en tiempo real, identificando el momento exacto en el que los ojos se cierran más de lo normal.

VI. Objetivos de arquitectura y comunicación

En lo que respecta a la estructura y conectividad del hardware, el diseño se orienta a crear dos servidores web independientes dentro del chip utilizando las funciones del entorno de desarrollo, destinando el puerto 81 para transmitir el video y el puerto 82 para recibir las órdenes de la alarma. Siguiendo esta idea, se configurará el programa de Python para que, al detectar que los ojos permanecen cerrados por más de dos segundos seguidos, cree un hilo de trabajo separado en la computadora encargado de enviar una petición rápida de internet hacia la ruta de la alarma. Como base física del sistema, se armarán las conexiones en la protoboard usando el adaptador serial FTDI rojo para subir el código y controlando la salida de energía del pin digital GPIO 2, lo que garantizará encender el buzzer físico de 5V por un tiempo exacto de 500 milisegundos sin retrasar el flujo del video.

VII. Objetivos de análisis y escalabilidad futura

Finalmente, enfocado en el análisis y el crecimiento futuro del proyecto, se tiene como meta medir y evaluar el tiempo de reacción de la red local, calculando los milisegundos que tardará la orden en viajar desde la computadora hasta que la tarjeta enciende la alarma. De manera simultánea, se analizará cómo afecta el tamaño del video al uso de la memoria de la tarjeta, buscando el punto de equilibrio ideal para que el servidor de video trabaje de forma estable y no se reinicie por falta de espacio. Como cierre de la propuesta, se proyecta la evolución del sistema hacia una pulsera inalámbrica portátil que incluya una tarjeta mini

ESP32 y el sensor del pulso MAX30102, transmitiendo los latidos del corazón por el aire directamente de chip a chip para combinar estos datos con la cámara y eliminar por completo las falsas alarmas.

VIII. Descripción estructural del sistema

El sistema electrónico de este proyecto se organiza mediante una arquitectura distribuida que divide el trabajo entre dos componentes principales conectados de forma inalámbrica a través de una red Wi-Fi local. El primer componente es la tarjeta del tablero (ESP32-CAM), la cual se encarga de la captura de imágenes y del control de la alarma física. Este bloque utiliza un sensor de video (OV2640) junto con un diodo LED infrarrojo para obtener imágenes claras en la oscuridad de la noche, y se encuentra conectada a la computadora mediante un adaptador serial FTDI de color rojo, el cual le provee energía y permite la comunicación de datos. La tarjeta tiene la función de actuar como un servidor web que transmite el video en vivo por el puerto 81 y activa un zumbador o buzzer de 5V conectado al pin digital GPIO 2 en el puerto 82, apenas recibe la orden de alerta.

El segundo componente es la estación de trabajo o computadora portátil, la cual asume la tarea de procesamiento y análisis inteligente de las imágenes de forma externa. Este bloque ejecuta un programa principal desarrollado en el lenguaje Python que se conecta de manera inalámbrica al servidor de video de la tarjeta. Mediante herramientas de análisis visual como MediaPipe de Google, la computadora calcula continuamente el nivel de apertura de los párpados del chofer. Si el programa detecta que los ojos permanecen cerrados por más de dos segundos seguidos, activa de inmediato un hilo de trabajo separado que envía una orden rápida por red hacia la tarjeta del tablero para encender la alarma sonora, logrando un control rápido y seguro sin interrumpir la transmisión de video.

IX. Diagrama de bloques del sistema

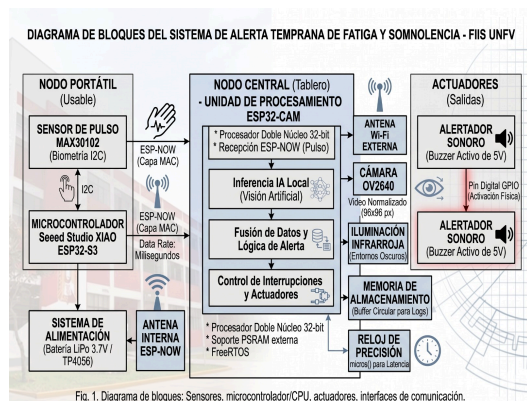


Fig. 1. Diagrama de bloques del sistema distribuido de alerta de fatiga: organización de los nodos portátiles y centrales, interfaces de comunicación inalámbrica y periféricos de salida física.

X. Plataforma seleccionada y justificación

Para el tablero central se ha seleccionado la tarjeta **ESP32-CAM**. Esta elección se justifica técnicamente porque tiene la capacidad de conectarse a redes Wi-Fi locales y un procesador de doble núcleo muy rápido, características necesarias que superan las limitaciones de los Arduinos comunes. La tarjeta permite trabajar con su memoria externa para guardar temporalmente las imágenes antes de transmitirlas. Al dejar el análisis de video pesado a una computadora a través de Python, aprovechamos el bajo costo y la resistencia de esta tarjeta para encargarse únicamente de la transmisión de red y del control físico de la alarma, logrando un sistema compartido, eficiente y seguro.

XI. Lista de métricas

Para validar científicamente el correcto funcionamiento y la viabilidad del sistema embebido, se medirán y analizarán tres métricas cuantitativas:

- **Tiempo de reacción de la alarma (ms):** Mide el tiempo total que toma mandar la orden desde Python a través de la red local hasta que la tarjeta física activa la energía de la alarma.
- **Estabilidad de la medición del ojo:** Evalúa si el programa reconoce bien la apertura del ojo (umbral 0.22) frente a los cambios de luz en la cabina usando

el LED infrarrojo, asegurando que el reloj distinga parpadeos normales de menos de medio segundo.

- **Velocidad de transmisión del video (FPS):** Mide el rendimiento de la red inalámbrica calculando cuántas fotos por segundo reales (Frames Per Second) procesa el programa de Python, garantizando un flujo continuo para que la detección sea inmediata.

XII. Plan de continuidad a IoT e IA

Este sistema se ha estructurado de forma modular, lo que garantiza que pueda crecer fácilmente hacia redes interconectadas y plataformas de análisis predictivo en los siguientes ciclos de la carrera. En la Fase de Integración a IoT (Internet de las Cosas), se contempla armar la pulsera inteligente portátil utilizando el sensor de pulso digital MAX30102 conectado a la tarjeta mini ESP32. Esta pulsera funcionará por sí sola alimentada por la batería pequeña de 3.7V y se podrá recargar por USB-C gracias al módulo de carga TP4056. Para evitar el uso de cables incómodos que molesten al conductor, ambas tarjetas ESP32 se comunicarán por el aire directamente usando el protocolo de bajo consumo ESP-NOW, mandando los datos del pulso en menos de un milisegundo.

En la Fase de Implementación de IA avanzada, se aprovechará la computadora con Python para combinar en paralelo las imágenes con el ritmo del corazón. Cuando una persona se está quedando dormida, las pulsaciones del corazón bajan de forma natural a un estado de reposo por debajo de los 60 latidos por minuto. Mediante una regla de decisión en el programa de Python, el sistema revisará ambas variables al mismo tiempo; si la cámara detecta los ojos cerrados y simultáneamente la pulsera reporta que el pulso cayó drásticamente, la alarma sonará con total seguridad. Al detectar que este cansancio se repite mucho en el viaje, la computadora se conectará a internet para mandar un aviso automático al Telegram de la empresa de transporte, permitiendo cambiar de chofer para descansar antes de que ocurra un accidente en la carretera.

XII. Cronograma de trabajo

➤ Fase N°1	243 horas	
➤ Fase de Inicio y Planificación	45 horas	
Análisis de la problemática de fatiga y somnolencia en choferes	1 hora	Yan;Roberto;Ricardo
Diseño de la arquitectura distribuida (PC y ESP32-CAM)	2 horas	Yan;Roberto;Ricardo
Definición de requerimientos técnicos del sistema	2 horas	Yan;Roberto;Ricardo
➤ Fase N°2	6 horas	
➤ Adquisición de Componentes y Materiales	6 horas	
Compra de la tarjeta de tablero (ESP32-CAM) y sensor de video OV2640	2 horas	Yan;Roberto;Ricardo
Compra de actuadores y componentes	2 horas	Yan;Roberto;Ricardo
Compra de materiales para la estructura física de soporte/tablero simulado	2 horas	Yan;Roberto;Ricardo
➤ Fase N°3	9 horas	
➤ Construcción Física de la Maqueta	9 horas	
Armado de conexiones en protoboard usando el adaptador serial FTDI rojo	4 horas	Yan;Roberto;Ricardo
Conexión y cableado del buzzer físico de 5V al pin digital GPIO 2	3 horas	Yan;Roberto;Ricardo
Montaje de la ESP32-CAM con el LED infrarrojo en la estructura del tablero simulado	2 horas	Yan;Roberto;Ricardo
➤ Fase N°4	9 horas	
➤ Desarrollo del Código y Lógica	9 horas	
Configuración de la cámara para capturar el rostro en resolución ligera	3 horas	Yan;Roberto;Ricardo
Programación del Servidor Web en el puerto 82 para recibir órdenes de alerta y activar el buzzer	2 horas	Ricardo;Roberto;Yan
Desarrollo del script en Python para recibir el video a través de la red Wi-Fi local	2 horas	Ricardo;Roberto;Yan
Programación del Servidor Web en el puerto 81 para transmitir el video en vivo	1 hora	Yan;Ricardo;Roberto
➤ Fase N°5	10 horas	
➤ Integración y Pruebas del Modelo	10 horas	
Integración de MediaPipe de Google para la detección de puntos faciales	3 horas	Ricardo;Roberto;Yan
Programación de la fórmula matemática de la Relación de Aspecto del Ojo	1 hora	Ricardo;Roberto;Yan
Configuración del hilo de trabajo separado en Python para enviar peticiones HTTP rápidas al detectar ojos cerrados por más de 2 segundos	1 hora	Yan;Roberto;Ricardo
Implementación del filtro que descarta una foto de cada dos para aliviar el procesador	1 hora	Roberto;Ricardo;Yan
➤ Finalización del Proyecto	5 horas	
Pruebas finales de funcionamiento continuo del sistema multimodal	2 horas	Yan;Roberto;Ricardo
Preparación de la demostración práctica del prototipo para la clase	3 horas	Yan;Roberto;Ricardo

Fig. 2. Cronograma de trabajo: Fases de planificación, adquisición de materiales, construcción e integración del sistema.

XIII. Arquitectura del microcontrolador y mapa de memoria

A. Organización física del procesador central

El núcleo de procesamiento seleccionado para el tablero de este proyecto es la tarjeta electrónica ESP32-CAM. Este dispositivo cuenta con un procesador Tensilica Xtensa de 32 bits con doble núcleo (Dual-Core) que opera a una velocidad de reloj de 240 MHz. A diferencia de las tarjetas tradicionales de un solo núcleo que ejecutan el código en una línea de tiempo estrictamente secuencial, esta arquitectura permite delegar tareas en paralelo.

En nuestro diseño de software distribuido, mientras un núcleo físico se encarga de sostener la pila de red inalámbrica Wi-Fi y transmitir el flujo de imágenes hacia la computadora, el segundo núcleo permanece libre para atender de forma asíncrona las solicitudes web de alerta y conmutar la energía del pin digital del buzzer sin generar congelamientos en el video

B. Distribución y mapa de la memoria interna (SRAM)

El microcontrolador dispone de **520 KB de memoria RAM estática (SRAM)** interna, la cual se organiza y mapea por el sistema operativo de la siguiente manera:

1. **Memoria de Datos Estáticos y Dinámicos (DRAM):** Se destina a almacenar las variables globales de nuestro programa (como los nombres de la red Wi-Fi y las contraseñas), las pilas de trabajo (*stacks*) de las funciones del servidor web, y los buffers de comunicación serial.
2. **Memoria de Instrucciones (IRAM):** Espacio donde el procesador aloja el código binario de las funciones críticas que deben ejecutarse a la máxima velocidad del reloj, como las rutinas de interrupción y los manejadores de eventos del Wi-Fi.

C. Uso estratégico de la memoria externa (PSRAM)

Debido a que el análisis visual requiere trabajar con imágenes, los 520 KB de la memoria SRAM interna resultan insuficientes y causarían un colapso del sistema (Crash) por falta de espacio. Por este motivo, nuestro código aprovecha de forma obligatoria los **4 MB de memoria RAM pseudo-estática externa (PSRAM)** que incluye la tarjeta.

Bajo esta configuración de hardware, cuando la función nativa `esp_camera_fb_get()` captura una foto desde el lente OV2640 en resolución QVGA, el paquete de bytes en formato JPEG se almacena temporalmente dentro de un buffer mapeado en la PSRAM externa. Con esta estrategia de diseño, la memoria SRAM interna queda completamente aliviada y protegida, asegurando que el chip pueda empaquetar y transmitir el video por el aire sin interrumpir los temporizadores de seguridad de la alarma. Con el fin de resumir las capacidades de almacenamiento físico detalladas anteriormente, se presenta la Tabla I.

Tabla I Capacidades de Memoria Interna y Externa del Microcontrolador

Componente de Memoria	Capacidad Física	Función Principal en el Proyecto
Memoria SRAM Interna	520 KB	Guarda variables globales, contraseñas de red y controla los hilos del servidor web.
Memoria PSRAM Externa	4 MB	Actúa como el buffer masivo donde se guardan temporalmente las fotos de la cámara antes de transmitirse.

Bajo esta perspectiva de administración de recursos, el flujo de datos e instrucciones dentro del microcontrolador se organiza de forma visual tal como se describe en la Fig. 3, evidenciando el camino que siguen las imágenes desde su captura física hasta su salida por la interfaz inalámbrica.

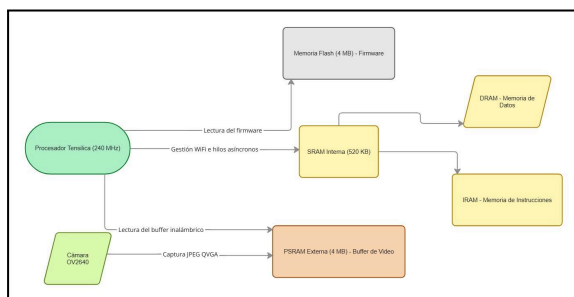


Fig. 3. Distribución lógica del mapa de memoria del microcontrolador y flujo de datos asíncrono para el almacenamiento temporal del flujo de video en la memoria PSRAM externa.

XIV. Interfaces de comunicación y programación de periféricos

A. Comunicación serial por cable (Interfaz UART)

Para hacer posible la transferencia inicial del programa y la visualización de los datos de control, el sistema utiliza la interfaz de

comunicación serial conocida como **UART (Universal Asynchronous Receiver-Transmitter)**. Físicamente, esta conexión se realiza uniendo los pines de transmisión (TX) y recepción (RX) de la tarjeta del tablero con el adaptador serial FTDI de color rojo, el cual se conecta por cable USB a la computadora.

Dentro del entorno de desarrollo de Arduino, esta interfaz se configura mediante la función nativa `Serial.begin(115200);`. Esta línea de código establece una velocidad física de transmisión de **115,200 baudios** (bits por segundo), permitiendo que el microcontrolador envíe mensajes rápidos de texto hacia la computadora para verificar que el inicio de los componentes ha sido exitoso.

B. Protocolo de red inalámbrica local (Sockets TCP/IP)

Con el propósito de enviar el video sin usar cables físicos incómodos dentro del vehículo, el sistema aprovecha el módulo Wi-Fi integrado en el chip de la tarjeta. En lugar de conectarse a internet, el microcontrolador se enlaza a una red inalámbrica local generada por un punto de acceso a través de las instrucciones de la librería `WiFi.h`.

Una vez establecida la conexión inalámbrica, el programa utiliza la biblioteca nativa `esp_http_server.h` para levantar dos servidores web independientes que atienden solicitudes de red de forma paralela en puertos distintos:

1. **Servidor de Video (Puerto 81):** Configura la ruta raíz del sistema mediante la estructura `httpd_uri_t stream_uri`. Al recibir una solicitud, ejecuta la función de control `stream_handler`, la cual toma de forma continua las imágenes capturadas por el lente de la cámara y las transmite por el aire hacia el programa de Python en la computadora usando ráfagas de paquetes JPEG.
2. **Servidor de la Alerta (Puerto 82):** Levanta de forma aislada la ruta `/buzzer` mediante la estructura `httpd_uri_t buzzer_uri`. Este puerto se queda escuchando la red de manera asíncrona; en el momento en que el programa de Python en la computadora detecta que el chofer tiene los ojos cerrados por más de dos segundos, envía una orden web rápida

(HTTP GET) hacia este puerto para disparar de inmediato el sonido de alerta.

C. Programación del actuador físico de alerta (GPIO)

El control físico del sonido de alerta se programa manipulando directamente los pines de entrada y salida de propósito general (GPIO) del microcontrolador. Dentro de la función de configuración inicial del código, se establece el pin **BUZZER_PIN** (definido físicamente en el pin digital **GPIO 2**) como una salida de energía mediante la instrucción `pinMode(BUZZER_PIN, OUTPUT);`.

Bajo esta perspectiva, cuando el servidor web del puerto 82 recibe la orden de alerta desde la computadora, el microcontrolador ejecuta de forma prioritaria la función de control `buzzer_handler`. Esta función pasa el pin a un estado alto con `digitalWrite(BUZZER_PIN, HIGH);` para enviar energía física al buzzer de 5V, y utiliza la instrucción de tiempo real `vTaskDelay(500/ portTICK_PERIOD_MS);` para sostener el sonido durante exactamente medio segundo antes de cortar la energía, asegurando un aviso rápido que despierta al conductor sin colgar el resto de las tareas inalámbricas del chip.

D. Lógica de inicialización y administración de tareas en tiempo real

Para garantizar la estabilidad del hardware y asegurar que los recursos de software se desplieguen sin saturar el sistema distribuido, el flujo del programa se organiza en fases consecutivas y bloques de control.

1) Fase 1 - Inicialización del Dispositivo

Esta primera etapa síncrona configura los parámetros iniciales y levanta los servicios inalámbricos dividiéndose en tres bloques secuenciales descritos en la Fig. 4:

- **Bloque 1 (Configuración de Periféricos y Memoria - Hardware Base):** Despierta las líneas de comunicación UART a 115,200 baudios, limpia el estado eléctrico de los pines GPIO y configura la reserva del buffer de video directamente dentro de la memoria PSRAM externa.
- **Bloque 2 (Inicialización del Sensor y Calibración Óptica):** El chip ejecuta la función nativa `esp_camera_init()`. Si

el microcontrolador valida que el sensor físico es un lente OV2640, accede a sus registros internos para calibrar de forma óptima los niveles de brillo y saturación. Ante cualquier fallo de hardware en esta comprobación, el sistema detiene su marcha y reporta el error por el puerto serial.

- **Bloque 3 (Conectividad Wi-Fi y Despliegue de Servidores HTTP):** La tarjeta activa su antena inalámbrica y se enlaza a la red local. Una vez obtenida la dirección IP, el firmware inicializa de forma asíncrona los dos servidores web en los puertos 81 (flujo multimedia) y 82 (escucha del buzzer).

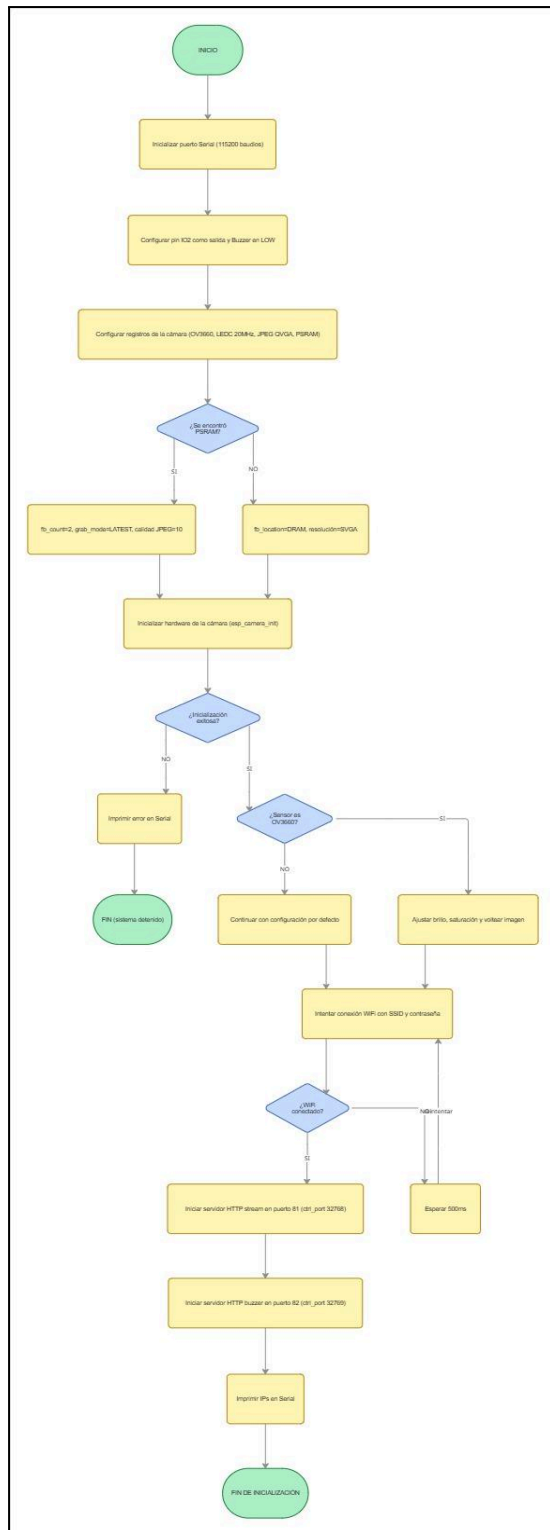


Fig. 4. Diagrama de flujo detallado para la Fase 1 de inicialización del dispositivo: (a) Bloque 1.1 de configuración de periféricos y asignación de memoria base, (b) Bloque 1.2 de inicialización del sensor óptico y calibración, (c) Bloque 1.3 de conectividad Wi-Fi y despliegue de servidores HTTP.

2) Fase 2 - Loop Principal / Idle State

Una vez completada de forma exitosa la inicialización de la Fase 1, el firmware de la tarjeta entra de manera automática en la Fase 2 (Loop Principal), detallada en la Fig. 5. Debido a que el sistema operativo de tiempo real FreeRTOS asume el control absoluto de las tareas de fondo de manera asincrónica, atendiendo en paralelo las ráfagas de imágenes por el puerto 81 y las interrupciones del buzzer por el puerto 82, el lazo principal de ejecución se mantiene intencionalmente libre de instrucciones pesadas. El procesador entra en un ciclo infinito de espera activa con un retardo de diez segundos (delay(10000ms)), manteniéndose en un estado de bajo consumo y despertando únicamente ante eventos o solicitudes HTTP entrantes por la red local.

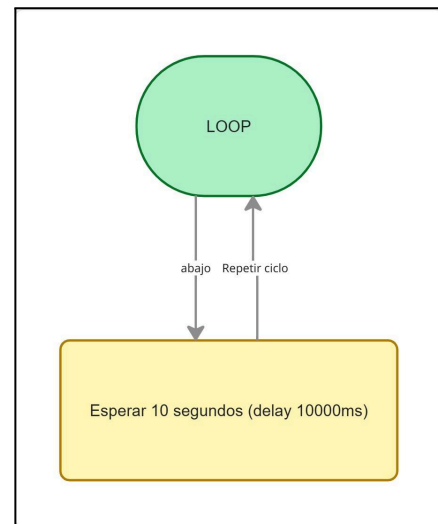


Fig. 5. Diagrama de flujo de la Fase 2 del programa, mostrando el estado de bajo consumo y reposo del lazo principal gestionado de forma asincrónica por FreeRTOS.

3) Fase 3 - Servidor de Streaming de Video (Puerto 81)

Cuando el programa principal de Python en la computadora inicia y realiza una petición de red HTTP GET hacia el puerto 81 de la tarjeta, el firmware levanta una tarea orientada a la transmisión masiva de datos descrita en la Fig. 6. El servidor acepta la conexión TCP e inyecta en la red una cabecera del tipo Multipart MJPEG.

Inmediatamente, el software entra en un bucle infinito condicionado a la actividad del socket inalámbrico. En cada ciclo, la instrucción nativa

esp_camera_fb_get() captura el fotograma binario almacenado temporalmente en la memoria PSRAM externa. Si el frame se obtiene con éxito, el sistema empaqueta la imagen enviando un delimitador de ráfaga (Boundary), la cabecera de longitud de bytes y el arreglo de datos JPEG directamente por el aire. Una vez transmitido, se invoca la función esp_camera_fb_return() para liberar de inmediato el búfer de memoria y evitar desbordamientos. Si el sistema detecta un error en el búfer óptico o una desconexión en el protocolo TCP, rompe de forma drástica el lazo de ejecución, cierra el socket y finaliza la tarea para proteger los recursos del procesador.

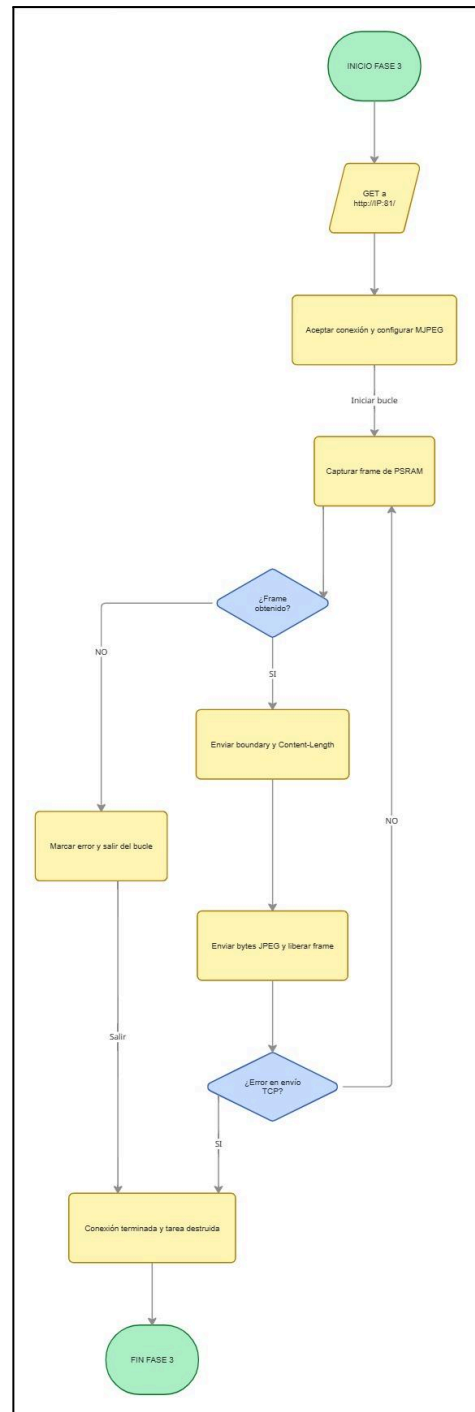
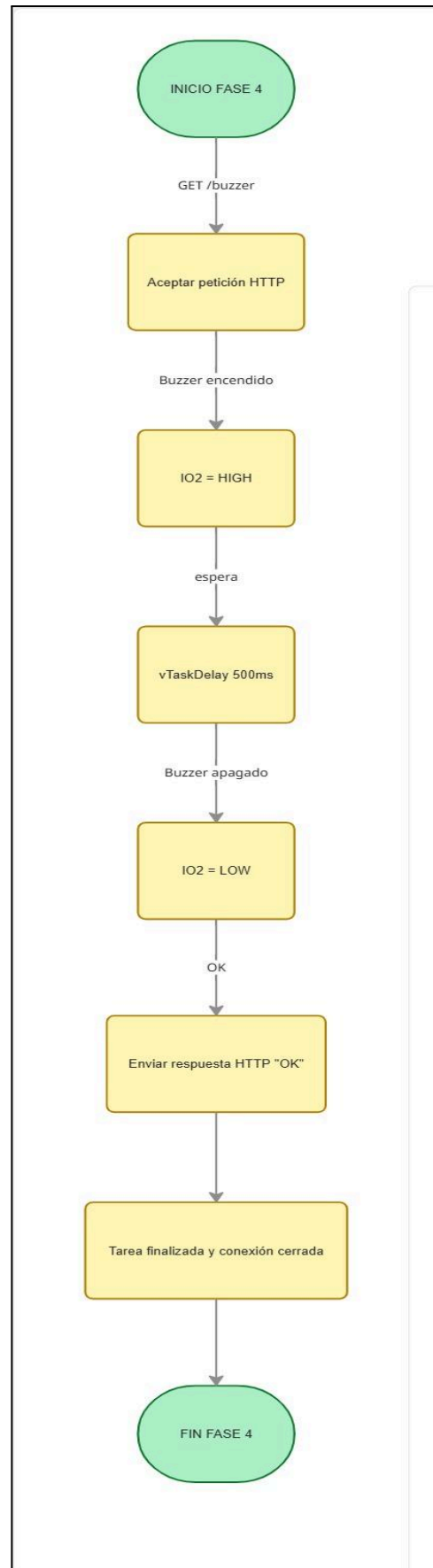


Fig. 6. Diagrama de flujo de la Fase 3 del programa, detallando el bucle continuo del servidor multimedia para el empaquetado y envío de tramas JPEG por red local TCP.

De manera totalmente independiente y en paralelo a la transmisión de video, la tarjeta ejecuta de forma asíncrona el manejador de la alarma sonora ante emergencias en la cabina, modelado en la Fig. 7. Este proceso se dispara como un evento síncrono en el momento en que un hilo de trabajo secundario en el script de Python detecta un microsueño e invoice una

petición HTTP GET hacia la ruta `/buzzer` en el puerto 82.

Al aceptar la solicitud web, el microcontrolador interrumpe de forma prioritaria sus funciones secundarias y modifica el estado del pin digital **BUZZER_PIN (GPIO 2)** conmutándolo a un nivel alto (HIGH). Esto satura físicamente de energía eléctrica al buzzer activo de 5V para encender la alarma de inmediato. Para sostener el sonido sin congelar los hilos de red del flujo multimedia, se ejecuta la instrucción de tiempo real `vTaskDelay(500 / portTICK)`, delegando una espera asíncrona de exactamente medio segundo al sistema operativo FreeRTOS. Cumplido este lapso de seguridad, el microcontrolador conmuta el pin a un estado bajo (LOW) para apagar el dispositivo acústico, despacha una respuesta HTTP con el mensaje de confirmación "OK" hacia la computadora y cierra la conexión, retornando al modo de escucha pasiva en la red local.



XV. Código fuente parcial comentado

A continuación, se presenta el firmware de control desarrollado para el nodo central del tablero. El programa configura la asignación de pines físicos de la cámara AI-Thinker, reserva los recursos en los búferes de memoria y levanta de forma asíncrona los dos servidores web en los puertos configurados:

```

1 #include "esp_camera.h"
2 #include <WiFi.h>
3 #include "esp_http_server.h"
4
5 // =====
6 // 1. PINES (AI-THINKER) - MAPEO DE HARDWARE BASE
7 // =====
8
9 #define PHDN_GPIO_NUM 32
10 #define RESET_GPIO_NUM -1
11 #define XCLK_GPIO_NUM 0
12 #define SIOD_GPIO_NUM 26
13 #define SIOC_GPIO_NUM 27
14 #define Y9_GPIO_NUM 35
15 #define Y8_GPIO_NUM 34
16 #define Y7_GPIO_NUM 39
17 #define Y6_GPIO_NUM 36
18 #define Y5_GPIO_NUM 21
19 #define Y4_GPIO_NUM 19
20 #define Y3_GPIO_NUM 18
21 #define Y2_GPIO_NUM 5
22 #define VSYNC_GPIO_NUM 25
23 #define HREF_GPIO_NUM 23
24 #define PCLK_GPIO_NUM 22
25 #define BUZZER_PIN 2 // Pin GPIO 2 enlazado al buzzer de 5V
26
27 const char* ssid = "MOVISTAR_CF60";
28 const char* password = "Fy5f7BFKxN4fdghNtvt";
29
30 httpd_handle_t stream_httpd = NULL;
31 httpd_handle_t buzzer_httpd = NULL;
32

```

```

33 // =====
34 // 2. FASE 3: STREAM HANDLER (PUERTO 81)
35 // =====
36
37 #define PART_BOUNDARY "123456789000000000000007654321"
38 static const char* STREAM_CONTENT_TYPE = "multipart/x-mixed-replace;boundary= \"PART_BOUNDARY\"";
39 static const char* STREAM_BOUNDARY = "\r\n-- \"PART_BOUNDARY\" \r\n\r\n";
40 static const char* STREAM_PART      = "Content-Type: image/jpeg\r\nContent-Length: %u\r\n\r\n";
41
42 static esp_err_t stream_handler(httpd_req_t *req) {
43     camera_fb_t* fb = NULL;
44     esp_err_t res = ESP_OK;
45     size_t _jpg_buf_len = 0;
46     uint8_t* _jpg_buf = NULL;
47     char part_buf[64];
48
49     res = httpd_resp_set_type(req, STREAM_CONTENT_TYPE);
50     if (res != ESP_OK) return res;
51     httpd_resp_send_hdr(req, "Access-Control-Allow-Origin: ***");
52
53     // Bucle infinito condicionado a la actividad del socket TCP inalámbrico
54
55     while (true) {
56         fb = esp_camera_fb_get(); // Captura el fotograma desde la memoria PSRAM
57         if (!fb) {
58             res = ESP_FAIL;
59         } else {
60             _jpg_buf_len = fb->len;
61             _jpg_buf = fb->buf;
62         }
63
64         // Envío secuencial de delimitadores, cabeceras y bytes binarios del JPEG
65
66         if (res == ESP_OK) res = httpd_resp_send_chunk(req, STREAM_BOUNDARY, strlen(STREAM_BOUNDARY));
67         if (res == ESP_OK) {
68             size_t hlen = sprintf(part_buf, 64, STREAM_PART, _jpg_buf_len);
69             res = httpd_resp_send_chunk(req, part_buf, hlen);
70         }
71         if (res == ESP_OK) res = httpd_resp_send_chunk(req, (const char*)_jpg_buf, _jpg_buf_len);
72
73         if (fb) {
74             esp_camera_fb_return(fb); // Libera de inmediato el búfer para evitar colapsos
75             fb = NULL;
76             _jpg_buf = NULL;
77         }
78         if (res != ESP_OK) break; // Rompe el bucle si detecta desconexión TCP
79     }
80     return res;
81 }

```

```

83 // =====
84 // 3. FASE 4: BUZZER HANDLER (PUERTO B2)
85 // =====
86 static esp_err_t buzzer_handler(httpd_req_t *req) {
87     // digitalWrite(BUZZER_PIN, HIGH); // Satura eléctricamente el pin GPIO 2 (Enciende)
88     vTaskDelay(500 / portTICK_PERIOD_MS); // Retardo asíncrono en FreeRTOS por 500 ms
89     digitalWrite(BUZZER_PIN, LOW); // Corta la energía del pin (Apaga)
90     httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "");
91     httpd_resp_send(req, "OK", 2); // Envía confirmación HTTP al cliente Python
92     return ESP_OK;
93 }
94
95 // =====
96 // 4. BLOQUE 3: INICIO DE SERVIDORES HTTP
97 // =====
98 void start(CameraServer()) {
99     httpd_config_t config1 = HTTPD_DEFAULT_CONFIG1();
100     httpd.server_port = B1; // Puerto multimedia dedicado al streaming de video
101     config1.ctrl_port = B2788;
102
103     httpd_uri_t stream_uri = {
104         .uri = "/",
105         .method = HTTP_GET,
106         .handler = stream_handler,
107         .user_ctx = NULL
108     };
109
110     if (httpd_start(&stream_httpd, &config1) == ESP_OK) {
111         httpd_register_uri_handler(&stream_httpd, &stream_uri);
112         Serial.println("Servidor stream OK en puerto B2");
113     } else {
114         Serial.println("ERROR: fallo servidor stream puerto B1");
115     }
116
117     httpd_config_t config2 = HTTPD_DEFAULT_CONFIG2();
118     config2.server_port = B2; // Puerto de control dedicado al actuador físico
119     config2.ctrl_port = B2789;
120
121     httpd_uri_t buzzer_uri = {
122         .uri = "/buzzer",
123         .method = HTTP_GET,
124         .handler = buzzer_handler,
125         .user_ctx = NULL
126     };
127
128     if (httpd_start(&buzzer_httpd, &config2) == ESP_OK) {
129         httpd_register_uri_handler(&buzzer_httpd, &buzzer_uri);
130         Serial.println("Servidor buzzer OK en puerto B2");
131     } else {
132         Serial.println("ERROR: fallo servidor buzzer puerto B2");
133     }
134 }

```

```

136 // *****
137 // 5. BLOQUES 1 Y 2: SETUP E INICIALIZACIÓN
138 // *****
139 void setup() {
140   Serial.begin(115200); // Inicializa comunicación serial UART
141   Serial.setDebugOutput(true);
142
143   pinMode(BUZZER_PIN, OUTPUT); // Configura pin del actuador como salida digital
144   digitalWrite(BUZZER_PIN, LOW); // Asegura que la alarma inicie apagada
145
146   // Asignación de registros y estructuras para el hardware de la cámara
147   camera_config_t config;
148   config.ledc_channel = LEDC_CHANNEL_0;
149   config.ledc_timer = LEDC_TIMER_0;
150   config.pin_d0 = V2_GPIO_NUM;
151   config.pin_d1 = V3_GPIO_NUM;
152   config.pin_d2 = V4_GPIO_NUM;
153   config.pin_d3 = V5_GPIO_NUM;
154   config.pin_d4 = V6_GPIO_NUM;
155   config.pin_d5 = V7_GPIO_NUM;
156   config.pin_d6 = V8_GPIO_NUM;
157   config.pin_d7 = V9_GPIO_NUM;
158   config.pin_xclk = XCLK_GPIO_NUM;
159   config.pin_pclk = PCLK_GPIO_NUM;
160   config.pin_vsync = VSYNC_GPIO_NUM;
161   config.pin_href = HREF_GPIO_NUM;
162   config.pin_sccb_sda = I2C0_GPIO_NUM;
163   config.pin_sccb_scl = I2C0_GPIO_NUM;
164   config.pin_pwdn = PWDN_GPIO_NUM;
165   config.pin_reset = RESET_GPIO_NUM;
166   config.xclk_freq_hz = 20000000;
167   config.pixel_format = PIXFORMAT_JPEG;
168   config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
169   config.fb_location = CAMERA_FB_IN_PSRAM; // Reserva buffer en memoria externa
170   config.jpeg_quality = FRAMESIZE_QVGA; // Resolución ligera de 320x240 píxeles
171   config.jpeg_quality = 12;
172   config.fb_count = 1;
173
174   // Validación dinámica de la presencia de memoria RAM pseudo-estática (PSRAM)
175   if (psramFound()) {
176     config.jpeg_quality = 10;
177     config.fb_count = 2; // Doble buffer para paralelismo de red
178     config.grab_mode = CAMERA_GRAB_LATEST;
179   } else {
180     config.frame_size = FRAMESIZE_SVGA;
181     config.fb_location = CAMERA_FB_IN_DRAM;
182   }
183
184   esp_err_t err = esp_camera_init(&config);
185   if (err != ESP_OK) {
186     Serial.printf("ERROR cámara: 0x%x\n", err);
187     return;
188   }
189 }

```

```

190 // Calibración y modificación de registros internos del lente OV2640
191 sensor_t* s = esp_camera_sensor_get();
192 if (s != NULL) {
193   if (s->id.PID == OV2640_PID) {
194     s->set_vflip(s, 0);
195     s->set_brightness(s, 1); // Ajuste óptico para entornos nocturnos oscuros
196     s->set_saturation(s, 0);
197   }
198   s->set_framesize(s, FRAMESIZE_QVGA);
199 }
200
201 WiFi.begin(ssid, password); // Conexión inalámbrica a la red local
202 WiFi.setSleep(false);
203 while (WiFi.status() != WL_CONNECTED) {
204   delay(500);
205   Serial.print(".");
206 }
207
208 startCameraServer(); // Despliegue de los sockets HTTP
209
210 Serial.println("\n Servidor activo");
211 Serial.print("Stream : http://");
212 Serial.print(WiFi.localIP());
213 Serial.println(":81/");
214 Serial.print("Buzzer : http://");
215 Serial.print(WiFi.localIP());
216 Serial.println(":82/buzzer");
217 }
218
219 // *****
220 // 6. FASE 2: LOOP PRINCIPAL / IDLE STATE
221 // *****
222 void loop() {
223   // El lazo principal reposa de forma indefinida delegando las tareas a FreeRTOS
224   delay(10000);
225 }

```

B. Script de procesamiento analítico visual (Python)

A continuación, se detalla el programa principal desarrollado en Python para ejecutarse en la computadora de procesamiento. El script decodifica la ráfaga de bytes MJPEG recibida por red, calcula matemáticamente el EAR y activa un hilo asíncrono de alerta ante riesgos críticos:

```

1 import cv2
2 import mediapipe as mp
3 import numpy as np
4 import time
5 import requests
6 import threading
7
8 # *****
9 # 1. CONFIGURACIÓN DE RED Y PARÁMETROS GEOMÉTRICOS
10 # *****
11 URL_ESP32 = "http://192.168.1.38:81/"
12 URL_BUZZER = "http://192.168.1.38:82/buzzer"
13
14 UMBRAL_EAR = 0.22 # Umbral límite para evaluar la oclusión ocular
15 TIEMPO_ALERTA = 2.0 # Temporizador crítico en segundos para microsuños
16
17 # *****
18 # 2. INICIALIZACIÓN DE LA MALLA FACIAL DE MEDIAPIPE
19 # *****
20 mp_face_mesh = mp.solutions.face_mesh
21 face_mesh = mp_face_mesh.FaceMesh(
22     max_num_faces=1,
23     refine_landmarks=True,
24     min_detection_confidence=0.5,
25     min_tracking_confidence=0.5
26 )
27
28 # Índices de los puntos de referencia faciales para los contornos de los ojos
29 OJO_IZQUIERDO = [362, 385, 387, 263, 373, 380]
30 OJO_DERECHO = [33, 160, 158, 133, 153, 144]
31
32 # *****
33 # 3. FUNCIÓN DE CÁLCULO MATEMÁTICO DEL EAR
34 # *****
35 def calcular_ear(puntos_ojo, landmarks, ancho, alto):
36     # Traduce las coordenadas relativas flotantes a píxeles absolutos en la RAM
37     coords = [
38         np.array([landmarks[p].x * ancho, landmarks[p].y * alto])
39         for p in puntos_ojo
40     ]
41     # Distancias Euclidianas verticales y horizontales de la geometría ocular
42     d_vertical1 = np.linalg.norm(coords[1] - coords[5])
43     d_vertical2 = np.linalg.norm(coords[2] - coords[4])
44     d_horizontal = np.linalg.norm(coords[8] - coords[3])
45     # Fórmula matemática de la Relación de Aspecto del Ojo
46     return (d_vertical1 + d_vertical2) / (2.0 * d_horizontal)
47

```

```

48 # *****
49 # 4. DISPARO ASÍNCRONO DEL ACTUADOR DE ALERTA
50 # *****
51 buzzer_activo = False
52
53 def activar_buzzer():
54     global buzzer_activo
55     if buzzer_activo:
56         return
57     buzzer_activo = True
58     try:
59         # Petición rápida de internet HTTP GET hacia la ruta de la alarma física
60         requests.get(URL_BUZZER, timeout=2)
61     except:
62         pass
63     buzzer_activo = False
64
65 # *****
66 # 5. CONEXIÓN INALÁMBRICA AL FLUJO MULTIMEDIA
67 # *****
68 print(" Conectando al ESP32-CAM...")
69 try:
70     stream = requests.get(URL_ESP32, stream=True, timeout=10)
71 except Exception as e:
72     print(f" No se pudo conectar: {e}")
73     exit()
74
75 print(" Conectado - iniciando detección...")
76
77 bytes_buffer = b""
78 contador_tiempo = 0
79 ojos_cerrados = False
80 frame_count = 0 # Contador de descarte de cuadros
81

```

```

82 # *****
83 # 6. LAZO PRINCIPAL DE PROCESAMIENTO ANALÍTICO
84 # *****
85 for chunk in stream.iter_content(chunk_size=4096):
86     bytes_buffer += chunk
87
88 # Identificación de delimitadores de compresión en el búfer de bytes
89 inicio = bytes_buffer.find(b'\xff\xd8') # Inicio binario de foto JPEG
90 fin = bytes_buffer.find(b'\xff\xd9') # Fin binario de foto JPEG
91
92 if inicio == -1 or fin == -1:
93     continue
94
95 jpg = bytes_buffer[inicio:fin + 2]
96 bytes_buffer = bytes_buffer[fin + 2:]
97
98 # FILTRO DE OPTIMIZACIÓN: Descarta un cuadro de cada dos para aliviar la CPU central
99 frame_count += 1
100 if frame_count % 2 == 0:
101     continue
102
103 frame = cv2.imdecode(np.frombuffer(jpg, dtype=np.uint8), cv2.IMREAD_COLOR)
104 if frame is None:
105     continue
106
107 alto, ancho, _ = frame.shape
108 frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
109 resultados = face_mesh.process(frame_rgb)
110

```

```

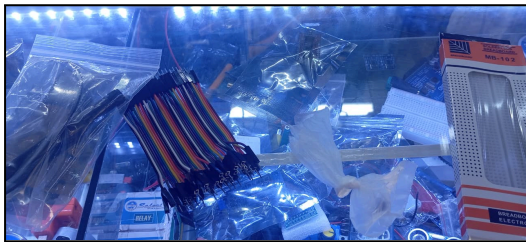
111 if resultados.multi_face_landmarks:
112     for face_landmarks in resultados.multi_face_landmarks:
113         # Cálculo del nivel de apertura promedio de los párpados
114         ear_izq = calcular_ear(OJO_IZQUIERDO, face_landmarks.landmark, ancho, alto)
115         ear_der = calcular_ear(OJO_DERECHO, face_landmarks.landmark, ancho, alto)
116         ear_promedio = (ear_izq + ear_der) / 2.0
117
118 # Evaluación de riesgo temporal continua frente al umbral límite
119 if ear_promedio < UMBRAL_EAR:
120     if not ojos_cerrados:
121         contador_tiempo = time.time() # Dispara el cronómetro de seguridad
122         ojos_cerrados = True
123     else:
124         tiempo_transcurrido = time.time() - contador_tiempo
125         if tiempo_transcurrido > TIEMPO_ALERTA:
126             # Dibujo del recuadro rojo de emergencia en la interfaz visual
127             cv2.rectangle(frame, (0, 0), (ancho, alto), (0, 0, 255), 10)
128             cv2.putText(frame, "¡¡¡ ALERTA: MICROSUÑO !!!",
129                 (ancho // 2 - 200, alto // 2),
130                 cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 3)
131             # Creación de un hilo de ejecución paralelo para no colgar el video
132             threading.Thread(target=activar_buzzer, daemon=True).start()
133         else:
134             ojos_cerrados = False # Reinicia el estado si el conductor parpadea normal
135
136 # Impresión del valor flotante en tiempo real dentro del monitor gráfico
137 color_ear = (0, 255, 0) if ear_promedio > UMBRAL_EAR else (0, 0, 255)
138 cv2.putText(frame, f"EAR: {ear_promedio:.2f}", (20, 40),
139             cv2.FONT_HERSHEY_SIMPLEX, 0.7, color_ear, 2)
140
141 cv2.imshow("IA - Monitor de Somnolencia", frame)
142 if cv2.waitKey(1) & 0xFF == ord('q'):
143     break
144
145 # *****
146 # 7. LIMPIEZA DE RECURSOS EN MEMORIA
147 # *****
148 cv2.destroyAllWindows()
149 print("Conexión finalizada.")

```

XVI. Evidencia de funcionamiento en tiempo real

Para validar de forma práctica el despliegue del sistema distribuido de alerta temprana, se recopilaron las evidencias de ejecución que abarcan desde la adquisición física de los componentes de hardware hasta las pruebas de software en entornos reales.

En primer lugar, la Fig. 8 presenta el registro fotográfico del equipo de trabajo durante la etapa de adquisición de materiales y componentes electrónicos en el mercado local. En esta evidencia se detalla la selección del microcontrolador ESP32-CAM, el lente óptico OV2640, el adaptador serial FTDI y el buzzer activo de 5V, asegurando la disponibilidad de los elementos base para el ensamblaje del tablero central y la posterior integración del modelo físico.



(a)



(b)

Fig. 8. Evidencia fotográfica del equipo de trabajo durante la adquisición y verificación de los componentes de hardware seleccionados para el prototipo.

Por otra parte, en la Fig. 9 se exponen los registros de diagnóstico capturados de forma directa desde el Monitor Serie del entorno Arduino IDE. En esta consola se evidencia el éxito de la Fase 1, detallando la correcta

inicialización del sensor de video OV2640, el enlace exitoso a la red inalámbrica local de la cabina y la asignación dinámica de las direcciones IP junto con la apertura de los sockets HTTP en los puertos 81 y 82.

```
Salida Monitor Serie X
Mensaje (Intro para mandar el mensaje de 'AI Thinker ESP32-CAM' a 'COM3')

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
config: 0, SPIWP:0xee
clk_drv:0x00, q_drv:0x00, d_drv:0x00, cs0_drv:0x00, hd_drv:0x00, wp_drv:0x00
mode:DIO, clock div:1
load:0x3fffc030,len:1344
load:0x40078000,len:13964
load:0x40080400,len:3600
entry 0x400805f0
...Servidor stream OK en puerto 81
Servidor buzzer OK en puerto 82

Servidor activo
Stream : http://192.168.1.38:81/
Buzzer : http://192.168.1.38:82/buzzer
```

Fig. 9. Captura de pantalla de los registros de inicialización y diagnóstico de red en el Monitor Serie de la tarjeta electrónica

